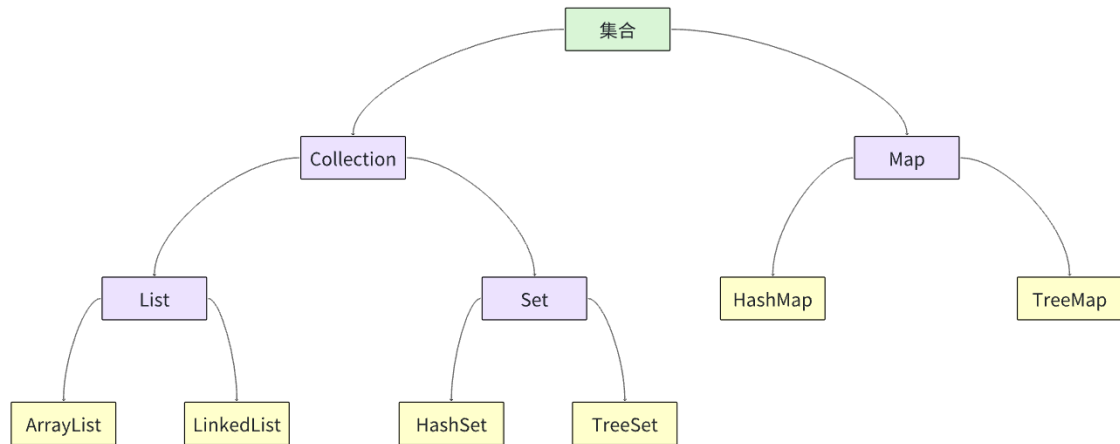


Task1. 集合与遍历



1. 请你简单了解一下上面的集合接口以及实现类，简单说一说他们各自的功能，并概括一下数组与集合的区别

- **Collection**是存一组元素的父接口
 - **List**有序、可重复的集合
 - **实现类:**
 - ArrayList:**查询快，增删相对慢
 - LinkedList:**增删快，查询相对慢
 - **Set:**无序,不可重复的集合
 - **实现类:**
 - HashSet:**去重效率高
 - TreeSet:**基于红黑树,能自动排序+去重
- **Map**是存储键值对关系的容器
 - **实现类:**
 - HashMap:**查找,插入效率高
 - TreeMap:**基于红黑树,按照key自动排序

2. 请你用增强for循环遍历list中的元素 依次打印出结果 给出你的代码和运行结果截图

```
for(Integer i : list){  
    System.out.println(i);  
}
```

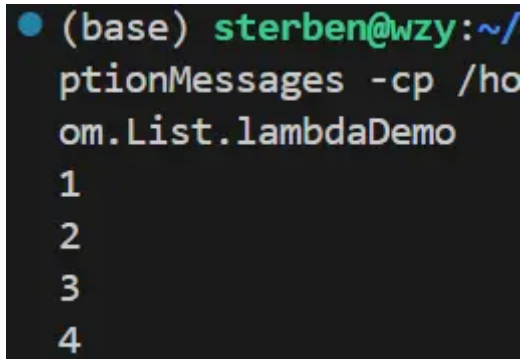
```
(base) sterben@wzy:~/scod-server/data/Use  
1  
2  
3  
4
```

3.

- 什么是匿名内部类

- 是一种**没有名字**的类,用来一次性**实现**接口或者**继承**父类
 - 什么是函数式接口?
 - 只有一个抽象方法的接口
- lambda表达式改写:**

```
list.forEach(i -> System.out.println(i));
```



```
(base) sterben@wzy:~/...  
ptionMessages -cp /ho...  
om.List.lambdaDemo  
1  
2  
3  
4
```

Task2. 泛型

```
//Repository接口  
package com.interface1;  
//要求能够储存任何类型的数据  
public interface Repository<T> {  
    int save(T e);  
    T getById(int id);  
}
```

```
//MyRepository实现  
package com.interface1;  
  
import java.util.HashMap;  
import java.util.Map;  
  
public class MyRepository<T> implements Repository<T> {  
    // 使用HashMap存储数据, key为ID, value为数据  
    private final Map<Integer, T> storage = new HashMap<>();  
    // 自增ID计数器, 从0开始  
    private int nextId = 0;  
  
    @Override  
    public int save(T e){  
  
        storage.put(nextId, e);  
        nextId++; // 自增ID计数器  
        return nextId - 1; // 返回新添加的元素的ID  
    }  
  
    @Override  
    public T getById(int id){  
        // 从HashMap中根据ID获取数据  
        return storage.get(id);  
    }  
  
    public void printAll(){
```

```

        if(storage.isEmpty()){
            System.out.println("仓库是空的");
        } else {
            for (Map.Entry<Integer, T> entry : storage.entrySet()) {
                System.out.println("ID: " + entry.getKey() + ", value: " +
                    entry.getValue());
            }
        }
    }
}

```

```

//Main测试类
package com.interface1;

public class Main {
    public static void main(String[] args) {
        MyRepository<String> stringRepository = new MyRepository<>();
        int id1 = stringRepository.save("Hello, world!");
        int id2 = stringRepository.save("拟好");
        int id3 = stringRepository.save("Java");
        stringRepository.printAll();

        MyRepository<Integer> integerRepository = new MyRepository<>();
        int id4 = integerRepository.save(99);
        int id5 = integerRepository.save(100);
        int id6 = integerRepository.save(101);
        integerRepository.printAll();

        MyRepository<User> userRepository = new MyRepository<>();
        int id7 = userRepository.save(new User("张三", 20));
        int id8 = userRepository.save(new User("李四", 25));
        int id9 = userRepository.save(new User("王五", 30));
        userRepository.printAll();
    }
}

```

```

ID: 0, Value: Hello, World!
ID: 1, Value: 拟好
ID: 2, Value: Java
ID: 0, Value: 99
ID: 1, Value: 100
ID: 2, Value: 101
ID: 0, Value: User{name='张三', age=20}
ID: 1, Value: User{name='李四', age=25}
ID: 2, Value: User{name='王五', age=30}

```

Task3. 面向对象设计模式

```
//Character接口
package com.Game;

public interface Character {
    void attack();
}
```

```
//Saber类
package com.Game;

public class Saber implements Character {
    private String name;
    private String skill;

    public Saber() {
        this.name = "阿尔托莉雅";
        this.skill = "Excalibur";
    }

    @Override
    public void attack() {
        System.out.println("[剑士] " + name + " 使用 " + skill + " 发动攻击!");
    }
}
```

```
//Archer类
package com.Game;

public class Archer implements Character {
    private String name;
    private String skill;

    public Archer() {
        this.name = "卫宫";
        this.skill = "Unlimited Blade works";
    }

    @Override
    public void attack() {
        System.out.println("[弓箭手] " + name + " 使用 " + skill + " 发动攻击!");
    }
}
```

```
//Caster类
package com.Game;

public class Caster implements Character {
    private String name;
    private String skill;

    public Caster() {
        this.name = "美狄亚";
        this.skill = "Rho Aias";
    }

    @Override
    public void attack() {
```

```

        System.out.println("[法师] " + name + " 使用 " + skill + " 发动攻击!");
    }

}

```

```

//角色工厂
package com.Game;
// TODO: 实现工厂方法，根据角色类型返回对应的角色对象
// 提示：如果传入未知类型，可以抛出 IllegalArgumentException
public class CharacterFactory {
    public static Character createCharacter(CharacterType type){
        switch (type) {
            case SABER:
                return new Saber();
            case ARCHER:
                return new Archer();
            case CASTER:
                return new Caster();
            default:
                throw new IllegalArgumentException("未知的角色类型: " + type);
        }
    }
}

```

```

//角色类型
package com.Game;

public enum CharacterType {
    SABER, ARCHER, CASTER
}

```

```

//测试类
package com.Game;

public class Test {
    public static void main(String[] args) {
        Character saber = CharacterFactory.createCharacter(CharacterType.SABER);
        saber.attack();
        Character archer =
CharacterFactory.createCharacter(CharacterType.ARCHER);
        archer.attack();
        Character caster =
CharacterFactory.createCharacter(CharacterType.CASTER);
        caster.attack();
        try{
            Character unknown = CharacterFactory.createCharacter(null); // 传入未知类型，抛出异常
            unknown.attack();
        } catch (IllegalArgumentException e) {
            System.out.println("创建角色时发生错误: " + e.getMessage());
        }
    }
}

```

```
[剑士] 阿尔托莉雅 使用 Excalibur 发动攻击!  
[弓箭手] 卫宫 使用 Unlimited Blade Works 发动攻击!  
[法师] 美狄亚 使用 Rho Aias 发动攻击!  
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "com.Game.CharacterType.ordinal()" because "type" is null  
    at com.Game.CharacterFactory.createCharacter(CharacterFactory.java:6)  
    at com.Game.Test.main(Test.java:11)
```